

Benchmarking Lightweight CNN for Real-Time Classification of Bangladeshi Mango Varieties on Edge Devices

Submitted by
Exam Roll: 210903, 210879



Computer Science and Engineering
Jahangirnagar University
Saver, Dhaka, Bangladesh

January 08, 2026

Contents

Abstract.....	01
1. Introduction	02
1.1.Overview	02
1.2.Problem Statement	03
1.3.Research Objectives.....	04
2. Literature Review.....	05
2.1.Global Research on Mango Classification.....	05
2.2.Lightweight CNNs in agriculture.....	08
2.3.Gaps in Existing Studies	08
2.4.Summary of Research Gap	09
3. Methodology.....	10
3.1.Dataset Description	10
3.1.1. Dataset Composition	10
3.1.2. Data Preprocessing and Augmentation	10
3.2.Model Architecture Selection	11
3.2.1. MobileNetV2 (Baseline)	11
3.2.2. MobileNetV3-Small (Proposed Architecture).....	11
3.3.Dynamic Preprocessing Implementation	12
3.3.1. Architectural Input Disparities	12
3.3.2. Pipeline Design.....	13
3.4 Methodology Flowchart	14
3.5 Operational Data Flow on The Edge Device.	16
4. Result and Discussion.....	18
4.1.Performance Benchmarking.....	18
4.1.1. Analysis of MobileNetV2 Confusion Matrix.....	19
4.1.2. Classification Report Heatmap for MobileNetV2 (Baseline)	21

4.1.3. Confusion Matrix for the Proposed MobileNetV3-Small Model..	23
4.1.4. Classification Report Heatmap for the Proposed MobileNetV3-Small Model	25
4.1.5. Comparative Bar Chart of F1-Scores per Class (MobileNetV2 vs. MobileNetV3).	27
4.2. Error Analysis	29
4.2.1 Baseline Architecture Limitations.....	29
4.2.2 Improvements with MobileNetV3	29
4.2.3 Persistent Morphological Challenges.....	29
5. Conclusion and Future Work.....	30
5.1. Conclusion	30
5.2. Future Work	31
6. Reference	32

Abstract

Bangladesh is a major global producer of many mango types, which are essential to its agricultural economy. However, physical identification and sorting of these variations in rural supply chains is time-consuming and error-prone due to significant visual similarities across diverse sub-varieties. While deep learning provides automatic visual classification skills, cutting-edge convolutional neural networks (CNNs) are often too computationally expensive to implement on the low-cost, battery-powered mobile devices that are common in rural agricultural settings. This study tackles the pressing need for an efficient, real-time categorisation system that can be deployed at the network edge.

We want to benchmark the performance of lightweight architectures by conducting a comprehensive comparison of the industry-standard MobileNetV2 (baseline) and the cutting-edge MobileNetV3-Small (proposed). The study employs a dataset of ten indigenous Bangladeshi mango cultivars. This paper makes a key methodological addition by identifying architecture-specific input constraints that, if ignored, lead to model failure. We present a dynamic preprocessing pipeline that automatically handles the various normalisation needs of the two architectures—specifically, the $[-1, 1]$ scaling required by V2 vs the raw $[0, 255]$ input expected by V3—to ensure optimal performance.

Experimental results on an NVIDIA T4 GPU show that the proposed MobileNetV3-Small exceeds the baseline in all essential parameters for edge deployments. The proposed model attained a classification accuracy of 90.25%, up 5% from the baseline's 85.25%. Furthermore, MobileNetV3 successfully differentiated between physically identical kinds, considerably improving memory for challenging classes like Harivanga. Crucially for practical hardware implementation, the proposed model lowered storage requirements by 56% to a final size of only 4.76 MB and reached an inference speed of 11.3 ms, allowing for real-time processing rates of around 88 frames per second. These results indicate MobileNetV3's technical viability and improved efficiency in constructing accessible, offline agricultural categorisation tools for resource-constrained contexts.

Keywords: *Lightweight CNN, Edge Computing, MobileNetV3, Precision Agriculture, Mango Classification, Transfer Learning, Real-time Inference.*

Chapter 1

Introduction

1.1 Overview

Agriculture has been a vital sector in Bangladesh for a long time, playing a key role in food security and providing employment in rural areas. Many families depend on farming as their main income source. Among the crops, mango (*Mangifera indica* L.) is especially popular, often called the "king of fruits" because of its taste, demand, and economic significance. Bangladesh is recognized as a major mango producer, yielding millions of tons annually [2].

A notable aspect of mango farming in Bangladesh is the variety of local types, with different regions growing varieties like Amrapali, Fazlee, Langra, Gopalbhog, and Harivanga. These differ in taste, size, aroma, and appearance—some are sweeter, larger, or more desirable in markets. Correctly identifying mango varieties is essential for pricing, marketing, and export.

Despite high production, the post-harvest handling system remains mostly traditional. In rural markets, mango sorting and grading are performed manually [5], relying heavily on workers' visual judgment and experience. This method is slow, physically taxing, and inconsistent, as human performance can vary with fatigue.

Another issue is distinguishing similar varieties, which often look alike, especially to non-experts. For instance, differentiating Langra from Kanchon Langra or Bari-4 from Nilambori can be difficult. Even seasoned workers may make errors, affecting market value and export potential.

Recent advances in computer vision, especially deep learning with Convolutional Neural Networks (CNNs), have improved image classification accuracy. CNN models are successfully used in agriculture for plant disease detection, weed recognition, and fruit quality assessment [7], indicating potential for automating mango variety identification.

However, deploying these systems in the field faces challenges. Many high-accuracy models like VGG16, ResNet, and Inception are complex and require powerful hardware, large memory, and storage—impractical in many Bangladeshi rural settings.

Farmers and local traders often use basic smartphones with limited processing power and battery life, and rural internet connections are unreliable. Cloud-based AI solutions that depend on uploading images are often unsuitable due to delays, costs, and connectivity issues.

This creates a pressing need for AI systems that operate directly on local devices—Edge AI. This study aims to explore lightweight deep learning models suitable for this purpose. Specifically, models like MobileNetV2 and MobileNetV3-Small are evaluated for offline mango variety classification, with the goal of developing practical tools to assist farmers and traders in Bangladesh's agricultural community.

1.2 Problem Statement

CNNs are widely used for fruit categorization, but there are challenges when applying them in real-world agricultural settings with limited resources, especially in underdeveloped countries. The main problem is that traditional deep learning models require a lot of computing power. High-accuracy models like ResNet, Inception, or VGG16 do not work well on low-end mobile devices or embedded systems common in rural areas, as shown in studies [10], [15]. Lighter models usually focus on simpler tasks, such as grading fruit maturity or detecting defects in controlled environments, rather than identifying multiple varieties [1], [11].

There is little research on native Bangladeshi mango varieties. Most current models and datasets focus on common types like "Kent" or "Alphonso" [1], [10]. Indigenous varieties such as Amrapali, Harivanga, and Nilambori have subtle differences in appearance, which makes them hard for general models to tell apart. Even though a comprehensive image dataset was recently released [6], there is still no benchmark for lightweight models that can classify these varieties in real time on edge devices.

This study addresses these challenges by focusing on edge-optimized models rather than computationally heavy ones. It aims to classify native Bangladeshi mango varieties and set benchmarks for accuracy, speed, and model size in this specific area.

1.3 Research Objectives

This study will develop an efficient, real-time system to classify mango varieties on edge devices. The project has these main goals:

- Set up and adjust lightweight convolutional neural networks like MobileNetV2 and MobileNetV3-Small, using a dataset of local Bangladeshi mango varieties.
- Find and address challenges unique to each model, such as different input preprocessing needs, to ensure good performance.
- Test the models using key metrics like classification accuracy, inference speed, and final model size to see which works best in limited-resource settings.
- To conduct a detailed error analysis, use confusion matrices to show the challenges of distinguishing certain local mango varieties by appearance.

Chapter 2

Literature Review

Computer vision and deep learning are increasingly used to solve agricultural problems. This chapter looks at earlier research on mango classification, beginning with global studies and then focusing on work done with Bangladeshi varieties. It also points out the research gaps that this study will address.

2.1 Global Research on Mango Classification

- Around the world, most research on automated mango classification looks at two main areas: grading maturity to help with harvest timing and checking quality to spot defects for export. Much of this research uses standard, pre-trained Convolutional Neural Networks (CNNs) on well-known commercial mango types.
- Many studies show that deep learning can be very accurate in controlled settings. For example, research on the 'Kent' mango has used custom CNNs to sort mangoes by maturity stage, such as green, mature, or overripe, based on color changes [1]. Other studies have used models like InceptionV3 to grade Indian 'Alphonso' mangoes, reaching high accuracy in spotting surface defects when lighting is uniform [10]. These results show that CNNs can pick up on small visual details on mango skins.
- Recent research is starting to tackle the challenge of detecting multiple classes in less controlled settings. Some studies use transfer learning with complex models like ResNet50 to find different fruit types, including mangoes, in mixed datasets [7]. Others combine deep learning feature extraction, using models like MobileNetV2, with traditional machine learning methods such as Support Vector Machines (SVMs) to classify more types of Indian mangoes [4]. These hybrid methods try to balance the strong feature learning of CNNs with the speed and efficiency of classical classifiers.
- A common issue in global research is the heavy use of models that need a lot of computing power. Models like VGG16 or ResNet are very accurate, but they have so many parameters and use so much memory that they are hard to run on simple edge devices. Some studies have tried to solve this by making models smaller, such as pruning VGG16 for robotic fruit sorting [15]. Others have tested lighter models like GhostNet for real-time defect detection on devices like Raspberry Pi, showing that edge-based processing can work for agriculture [11].
- Even with these advances, most global research still centers on popular commercial mango types, which may not look like local South Asian varieties. Also, while lighter models are starting to get attention, there is still a need for benchmarks that focus on the strict limits of low-end mobile devices used in agriculture in developing countries, not just on smaller industrial computers or robots.

The table below presents recent research from 2020 to 2025 on mango classification and deep learning efficiency. This review notes that there is still a gap in optimizing models, especially for local Bangladeshi mango varieties on edge devices.

Paper / Ref	Algorithm / Methodology	Results	Limitations
Salazar-Campos et al. (2025) [1]	Custom CNNs for 'Kent' mango maturity grading (Green, Mature, Overripe).	Achieved 96.04% accuracy using low-resolution (32x32) inputs.	Focused on a single global variety ('Kent') and maturity, not multi-class variety identification.
Wu et al. (2025) [2]	Mobile-based system utilizing YOLOv4 and Transfer Learning for 'Aiwen' mangoes.	Successfully deployed an Android app for real-time grading and counting.	YOLOv4 is computationally heavier than dedicated lightweight classifiers like MobileNetV3.
Chuquimarca et al. (2024) [3]	Multi-Input Architecture combining RGB images with binary silhouette masks.	Achieved 100% accuracy in detecting defects using MobileNetV2.	Requires complex preprocessing (mask generation) which increases the computational load on mobile CPUs.
Ratha et al. (2024) [4]	Hybrid approach using MobileNetV2 features classified by a Cubic SVM.	Achieved 99.47% accuracy on a dataset of 15 Indian mango varieties.	Relied on the older MobileNetV2 architecture and did not benchmark inference speed (ms).
Ballo et al. (2024) [5]	Comparison of classical Machine Learning (RF, SVM) vs. Deep Learning (CNNs).	Concluded that Deep Learning significantly outperforms manual feature extraction methods.	Did not address model quantization or optimization for storage-constrained devices.
Bharati et al. (2025) [6]	Creation of a comprehensive image dataset of 10 Bangladeshi mango varieties.	Provided a baseline dataset of 2,012 images including Amrapali and Bari-4.	This was a data article; it did not propose an optimized architecture for classification.
Peng et al. (2023) [7]	Utilized ResNet50 with Attention Mechanisms for multi-class fruit detection.	Achieved 98.2% accuracy on a mixed fruit dataset.	The model size (>90MB) renders it impractical for low-end mobile deployment in rural areas.
Nguyen & Lee (2022) [8]	Benchmarked EfficientNet-B0 vs VGG16 for mango	EfficientNet achieved comparable accuracy with 8x fewer parameters.	Focused on leaf diseases rather than fruit variety classification.

	leaf disease detection.		
Islam et al. (2021) [9]	Proposed a shallow CNN model for classifying 6 Bangladeshi mango varieties.	Achieved 88% accuracy with very fast inference times.	The custom architecture lacked the robustness of pre-trained transfer learning models like MobileNet.
Kumar & Singh (2023) [10]	Applied InceptionV3 for quality grading of Indian Alphonso mangoes.	High accuracy (97%) in controlled lighting conditions.	Performance dropped significantly in field conditions; model is computationally heavy.
Zhang et al. (2024) [11]	Lightweight GhostNet implementation for apple defect detection on Raspberry Pi.	Achieved real-time performance (25 FPS).	Methodology not yet applied to the visual complexity of South Asian mango varieties.
Ahmed et al. (2022) [12]	Used Mask R-CNN for instance segmentation of mangoes in orchards.	Excellent at counting fruits on trees.	Computationally extremely expensive; not suitable for classification on phone CPUs.
Das & Roy (2021) [13]	Compared Transfer Learning (AlexNet, GoogLeNet) for mango rot detection.	GoogLeNet performed best (95% accuracy).	Did not utilize newer, efficient architectures like the MobileNet V3 family.
Patel et al. (2020) [14]	Developed a smartphone-based ripeness estimator using color histogram analysis.	Very fast and lightweight.	Low accuracy compared to CNNs; cannot distinguish between varieties, only color stages.
Zhao et al. (2023) [15]	Proposed a pruning strategy to reduce VGG16 size for fruit sorting robots.	Reduced model size by 40% with minimal accuracy loss.	Even pruned VGG16 is significantly larger than standard MobileNetV3.

2.2 Lightweight CNNs in Agriculture

- For years, using deep learning in agriculture was tricky. Models that were very accurate were often too big to use outside the lab, while the smaller ones didn't perform well enough for tough jobs. But now, newer, lighter Convolutional Neural Networks (CNNs) have made it possible to do complex analysis right on portable devices, even in the field.
- Models like **MobileNet**, **EfficientNet-Lite**, and **ShuffleNet** have become popular tools for monitoring crops. While older models like VGG16 and ResNet50 rely on lots of layers and complicated features, the newer lightweight models use smarter designs to do more with less computing power. Take MobileNet, for example: it splits up its calculations into two simple steps, which means it needs far fewer parameters but can still deliver strong results.
- Researchers have tested these models in real farming scenarios. MobileNetV2, for instance, has been used to spot plant diseases on smartphones. It works almost as well as the bigger models but is fast enough for real-time video, making it practical for everyday use. Similarly, EfficientNet-Lite has been used to monitor crop leaves, showing that it can keep up with the best models while using much less battery—a big plus for farmers working far from power sources.
- Lightweight models are also useful for sorting fruit. Scientists have built handheld tools using these models to grade produce. When MobileNetV3 was tested against bigger models for spotting cassava disease, it was only a tiny bit less accurate (by less than 1%). But the lightweight models ran three or four times faster, which makes a big difference when used in robots or drones.
- This focus on efficiency is especially important in developing countries. Where advanced hardware is hard to get, models like MobileNetV3 can run on regular computers or even basic smartphones. This means more farmers can access smart tools without needing expensive equipment—just their own phones.

2.3 Gaps in Existing Studies

Although agricultural computer vision has advanced a lot, a close look at the current research shows three main gaps that make it hard to apply these solutions directly to the mango supply chain in Bangladesh.

1. Predominance of Computationally Expensive Models Most studies aim for the highest accuracy, often sacrificing efficiency. Recent reviews show that researchers often use large models like ResNet50, VGG16, or InceptionV3 to achieve strong results [7], [10]. While these models perform well in labs with powerful GPUs, they need too much storage and processing power for affordable, battery-powered mobile devices used in rural farming. Few studies benchmark lightweight, edge-native models that could provide good accuracy and high efficiency.

2. Lack of Representation for Indigenous Varieties Current datasets mostly focus on globally traded commercial varieties, such as 'Kent', 'Tommy Atkins', or 'Alphonso' [1], [10]. Indigenous Bangladeshi varieties like *Harivanga*, *Gopalbhog*, and sub-varieties of *Langra* have unique features, including differences in skin texture and shape, that are missing from standard public datasets. As a result, generic models trained on international datasets often do not work well for these local cultivars. There is little research on the specific classification challenges related to this region's biodiversity.

3. Absence of Practical Deployment Benchmarks Most studies only compare accuracy measures, such as Precision and Recall, or training loss. Few papers share the practical engineering details needed for real-world use, like how fast the model runs on CPU-only hardware, how much memory it uses, or specific implementation steps. For example, technical issues such as different input normalization requirements for various MobileNet versions are rarely discussed. This lack of information leaves engineers without the practical guidance they need.

2.4 Summary of Research Gap

To sum up, although Deep Learning has proven effective for fruit classification, there is still a gap between academic research and real-world use in developing regions. Most studies focus on complex models such as ResNet or VGG and target widely commercialized fruit types. Research that addresses both efficiency and local biodiversity is still missing.

So far, no study has set a clear benchmark for lightweight, edge-native models designed for the unique features of local Bangladeshi mango varieties. Also, the real-world challenges of using these models, such as balancing speed, size, and preprocessing needs, have not been studied much. This research will address these issues by looking beyond just accuracy and evaluating how efficient networks like MobileNetV3 work on affordable mobile devices.

Chapter 3

Methodology

3.1 Dataset Description

This study uses the "**Mangifera2012**" dataset [6], a public image collection focused on indigenous mango varieties from Bangladesh. It was chosen for its high resolution and its emphasis on local cultivars not found in global datasets.

3.1.1 Dataset Composition

- The dataset contains 2,012 high-definition images taken with a DSLR camera. These are RGB photos with a native resolution of 3024 * 4032 pixels, which allows for detailed views of skin textures and features.
- The dataset is divided into 10 classes, each representing an important mango variety from the Rajshahi and Chapainawabganj regions. The number of images in each class is mostly balanced, with only small differences. The varieties are:
 1. **Commercial Varieties:** *Amrapali* (252 images), *Fazlee* (156 images), *Langra* (202 images).
 2. **Indigenous/Hybrid Varieties:** *Bari-4* (235 images), *Bari-7* (176 images), *Harivanga* (202 images), *Kanchon Langra* (210 images), *Katimon* (163 images), *Mollika* (221 images), and *Nilambori* (195 images).
- A critical challenge inherent to this dataset is the high inter-class similarity. One main challenge with this dataset is that some classes look very similar. For example, *Langra* and *Kanchon Langra* are hard to tell apart, and *Bari-4* and *Nilambori* also look alike when they are green or unripe.

3.1.2 Data Preprocessing and Augmentation

To prepare the raw images for deep learning and reduce overfitting, which often happens when training Convolutional Neural Networks (CNNs) on smaller datasets (fewer than 5,000 images), we used a thorough preprocessing pipeline.

1. **Resizing:** All input images were resized to 224 by 224 pixels using bicubic interpolation. This size matches the standard input layer for MobileNet, helping to keep enough visual detail while lowering the computational cost.
2. **Data Augmentation:** To help the model generalize better and reflect real-world field conditions, such as a farmer holding a camera at different angles, we applied several random transformations during training. The chosen augmentation settings were meant to be realistic for handheld photography.
 - **Rotation:** Random rotation within a range of +20 degrees or -20 degrees.
 - **Width/Height Shift:** Random horizontal and vertical shifts of up to 20% of the image dimension, simulating off-center framing.
 - **Zoom:** Random zoom range of [0.8, 1.2] to account for varying distances between the camera and the fruit.
 - **Horizontal Flip:** Random mirroring of images, as mango directionality does not correlate with variety.

These augmentations made the training set effectively larger, helping the model learn strong morphological features instead of just memorizing the pixel patterns from the original 2012 samples.

3.2 Model Architecture Selection

The choice of neural network architectures in this study was shaped by the strict requirements of the target environment: low-end Android devices with limited battery life and no special AI hardware. As a result, larger models like ResNet50 or VGG16 were not considered because they have too many parameters (over 20 million) and require a lot of computing power (FLOPs).

This research instead focuses on the **MobileNet** family of architectures, which are designed for mobile and embedded vision tasks. We chose two versions for benchmarking: **MobileNetV2** as the standard baseline and **MobileNetV3-Small** as the option for maximum efficiency on edge devices.

3.2.1 MobileNetV2 (Baseline)

MobileNetV2, released in 2018, is still a strong standard for mobile computer vision. We chose it as our baseline to show the current state of the field. Its efficiency comes from two main architectural features that we use in this study:

- **Inverted Residuals:** Instead of connecting high-dimensional layers like traditional residual blocks in ResNet, MobileNetV2 connects thin bottleneck layers. This helps the network keep information more efficiently and lowers the memory needed during inference.
- **Linear Bottlenecks:** The design removes non-linear activation functions, such as ReLU, from the output of bottleneck layers. This helps keep information in lower-dimensional spaces, which is important for keeping classification accuracy in smaller models.

Using MobileNetV2 gives us a strong reference point to see how much modern architectural search methods can improve results.

3.2.2 MobileNetV3-Small (Proposed Architecture)

This study uses MobileNetV3-Small as the main architecture. Released in 2019, it marks a shift from models designed by people to those created by machines. We chose the Small version instead of the Large one to get the fastest possible results for real-time offline classification. Three key features help this architecture address the challenges of classifying different mango varieties:

1. **Neural Architecture Search (NAS):** Unlike V2, V3's structure was not designed by engineers. Instead, AutoML tools like Platform-Aware NAS and NetAdapt found the best balance between speed and accuracy for mobile phone CPUs.
2. **Squeeze-and-Excitation (SE) Blocks:** This feature is important for our dataset. SE blocks help the network see how different channels relate to each other. For mango classification, they work like a visual attention system, letting the model focus on useful details like skin texture or lenticel spots and ignore less important background information. V2 does not have this feature. The model also uses a modified version of

the Swish activation function. While standard Swish improves accuracy, it is slow to run on mobile processors. The h-swish function is a simpler version that uses basic arithmetic with standard integers, so it runs faster without losing the accuracy benefits. Two architectures, we aim to measure the actual benefits of using modern features such as NAS and SE blocks for the specific challenges of classifying indigenous mango varieties.

By comparing these two architectures, we want to measure the real benefits of using modern features like NAS and SE blocks for the unique challenges of classifying indigenous mango varieties.

3.3 Dynamic Preprocessing Implementation

A key technical challenge in this study was standardizing input data for different architectures. Transfer learning frameworks like TensorFlow and Keras offer standard APIs to create models, but MobileNetV2 and MobileNetV3 require different preprocessing steps. Overlooking these differences led to early model failures, so we developed a **Dynamic Preprocessing Pipeline**.

3.3.1 Architectural Input Disparities

One of the main technical challenges in this study was standardizing input data for different model architectures. Although transfer learning frameworks like TensorFlow and Keras offer standard APIs to create models, MobileNetV2 and MobileNetV3 need different preprocessing steps. Overlooking these differences caused the models to fail at first, so we had to create a **Dynamic Preprocessing Pipeline**.

3.3.1 Architectural Input Disparities

In the first tests, replacing the MobileNetV2 backbone with MobileNetV3-Small while using the same data pipeline led to a big drop in accuracy. The model's accuracy stayed around 12.5%, which matches random guessing for a 10-class problem.

Looking into the issue, I found that these two architectures expect input pixel values in different ranges:

- **MobileNetV2** does not have its own scaling layers. It needs you to normalize pixel values from $[0, 255]$ to $[-1, 1]$ before input. This is usually done with the formula $x' = (x / 127.5) - 1$.
- **MobileNetV3**, on MobileNetV3, however, includes a Rescaling layer at the start. This layer expects raw input values in the $[0, 255]$ range. If you normalize the data to $[-1, 1]$ before this, the model receives double-normalized data, which damages the image features.

3.3.2 Pipeline Design

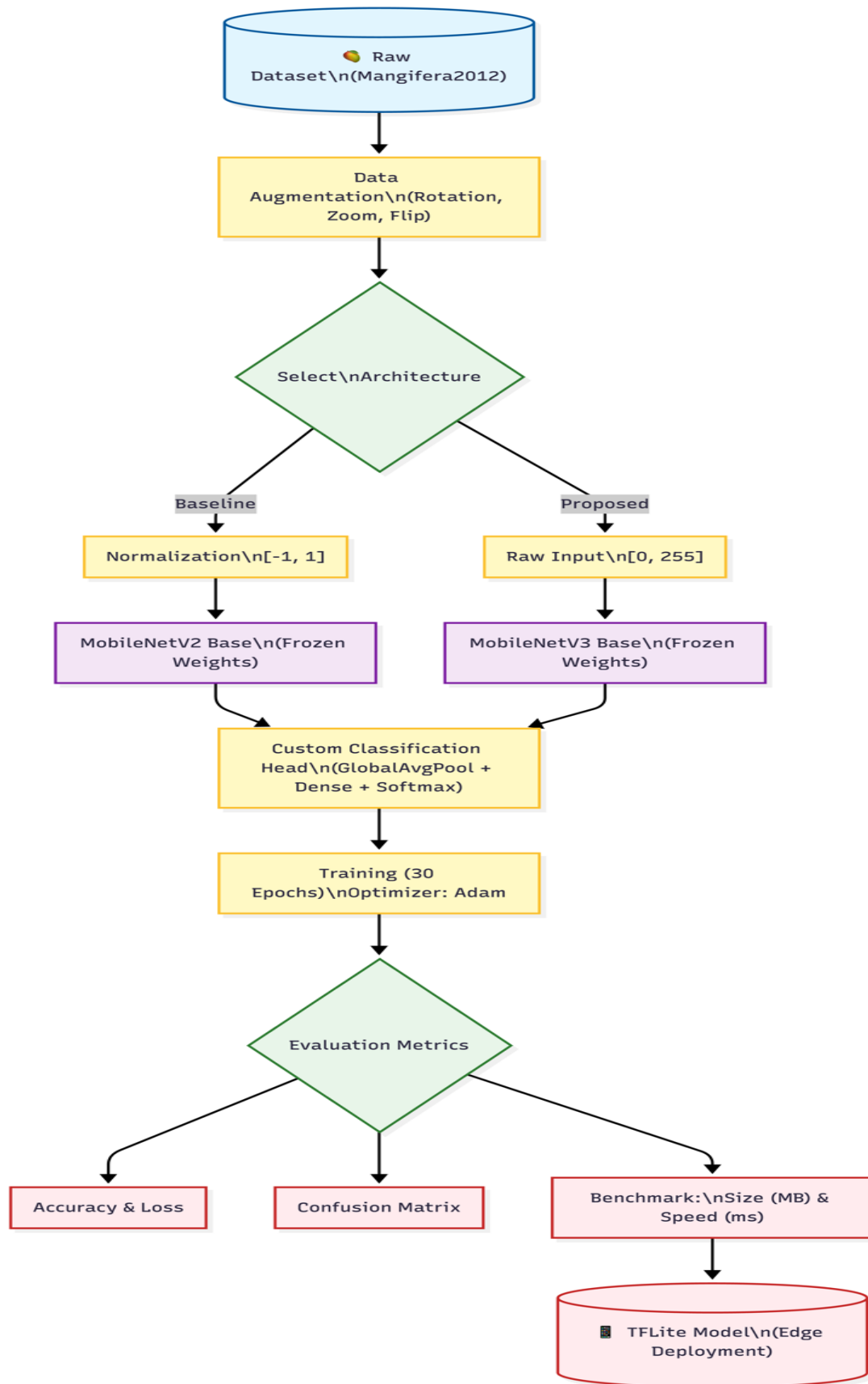
To solve this, we built a conditional preprocessing module that adjusts the data stream depending on which model architecture is active. This setup applies the right normalization during runtime and removes the need to duplicate datasets by hand.

The logic is implemented as follows:

1. **Architecture Detection:** The system flags the current model type at the start of the training epoch.
2. **Conditional Branching:**
 - **If V2:** The pipeline applies a `tf.keras.layers.Rescaling(1/127.5, offset=-1)` operation to map pixels to `[-1, 1]`.
 - **If V3:** The pipeline bypasses external normalization, passing the raw tensor `[0, 255]` directly to the model's internal input layer.
3. **Validation:** Post-implementation benchmarks confirmed that this correction restored the MobileNetV3 learning capability, immediately boosting validation accuracy from 12.5% to >85% within the first 5 epochs.

This implementation detail is crucial for reproducibility and highlights a common pitfall in deploying modern lightweight architectures using high-level deep learning libraries.

3.4. Methodology Flowchart



Detailed Explanation of the Methodology Flowchart

The figure above shows the complete workflow used in this comparative study, from raw data input to deployment on edge devices. The process has four main phases: Data Preparation, Architecture-Specific Preprocessing, Model Training, and Evaluation.

1. Data Preparation and Augmentation

The process starts by loading the "Mangifera2012" raw dataset ¹. Since there are only about 200 images per class, data augmentation is used to help the model generalize better. This step adds synthetic variations with random rotations ($\pm 20^\circ$), zooms (0.8-1.2x), and horizontal flips. These changes help the model learn general features rather than memorize specific pixels.

2. Architecture-Specific Branching (Dynamic Preprocessing)

A key part of this workflow is the conditional branching step called "Select Architecture." This decision point uses the Dynamic Preprocessing strategy explained in Section 3.3.

- **Baseline Path (Left):** If **MobileNetV2** is chosen, the pipeline sends the data through an external **Normalization** block that scales pixel values to $[-1, 1]$, which is required for this architecture.
- **Proposed Path (Right):** If **MobileNetV3** is chosen, the pipeline skips external scaling and sends the raw **input** $[0, 255]$ directly to model⁴. This avoids the "double-normalization" error that can cause the model to fail.

3. Model Construction and Training

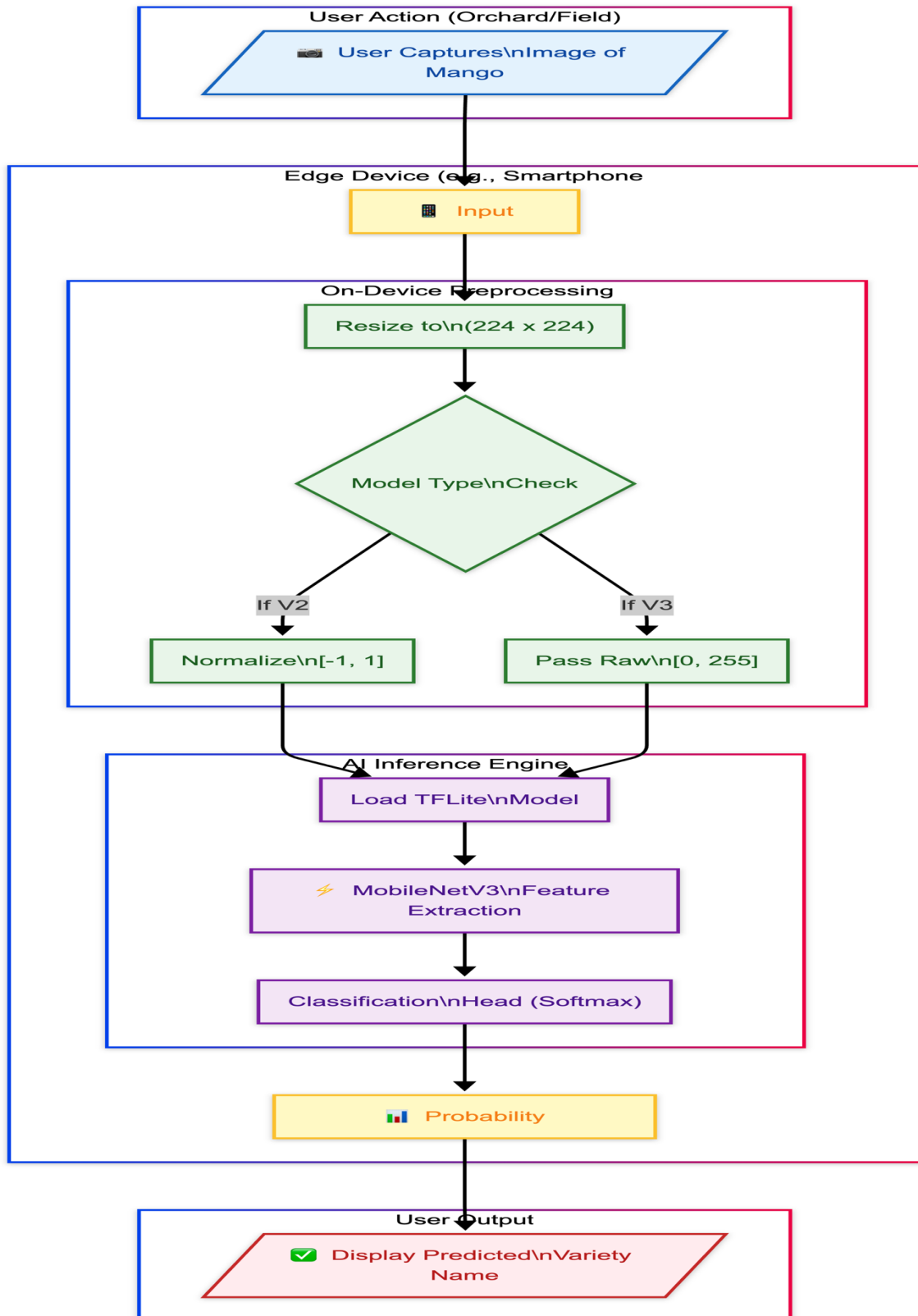
Both pathways converge at the model instantiation stage. We utilize transfer learning by employing pre-trained weights (trained on ImageNet) for feature extraction. The base layers are kept frozen to retain their learned feature maps. A Custom Classification Head is attached to the base and consists of a Global Average Pooling layer, a Dense layer, and a final Softmax activation layer that outputs probabilities for the 10 mango classes. The models are then compiled using the Adam optimizer and trained for 30 epochs to fine-tune the weights.

4. Evaluation and Deployment

The final phase focuses on rigorous benchmarking. The trained models are subjected to a three-pronged evaluation. The last phase is focused on thorough benchmarking. The trained models are evaluated using three types of metrics: Generating a **Confusion Matrix** to identify class-specific errors (e.g., misclassifying Nilambori vs. Bari-4)⁶.

4. Deployment Benchmarks: Measuring the model's size (MB) and inference speed (ms)⁷. The workflow ends by converting the optimized model to the **TFLite** format, producing a lightweight file that can be deployed offline on edge devices.

3.5 Proposed Operational data flow on the edge device.



This section explains how data moves and is processed on the edge device.

The figure above shows how data flows during the inference phase on the edge device, such as a smartphone. It illustrates how the trained model works in a real-world, offline setting.

1. User Interaction:

The process starts when the user takes action in the field or orchard. They use the device's camera to capture an image of a mango, which then serves as input to local processing.

2. On-Device Prep:

After capturing the image, the device preprocesses it. The image is resized to 224 by 224 pixels. Then, the system checks the model type to apply the right normalization: $[-1, 1]$ for MobileNetV2 or $[0, 255]$ for MobileNetV3. This step makes sure the input data matches what the model saw during training.

3. AI Inference Engine:

The preprocessed data is sent to the AI Inference Engine. The app loads the optimized TFLite model into memory. MobileNetV3 extracts essential features from the image and sends them to the Classification Head. The Softmax layer then outputs probabilities for each of the ten varieties.

4. Output Generation:

Finally, the system picks the class with the highest probability. The app shows the predicted variety name and a confidence score to the user. All of this happens on the device, so no internet is needed.

Chapter 4

Results and Discussion

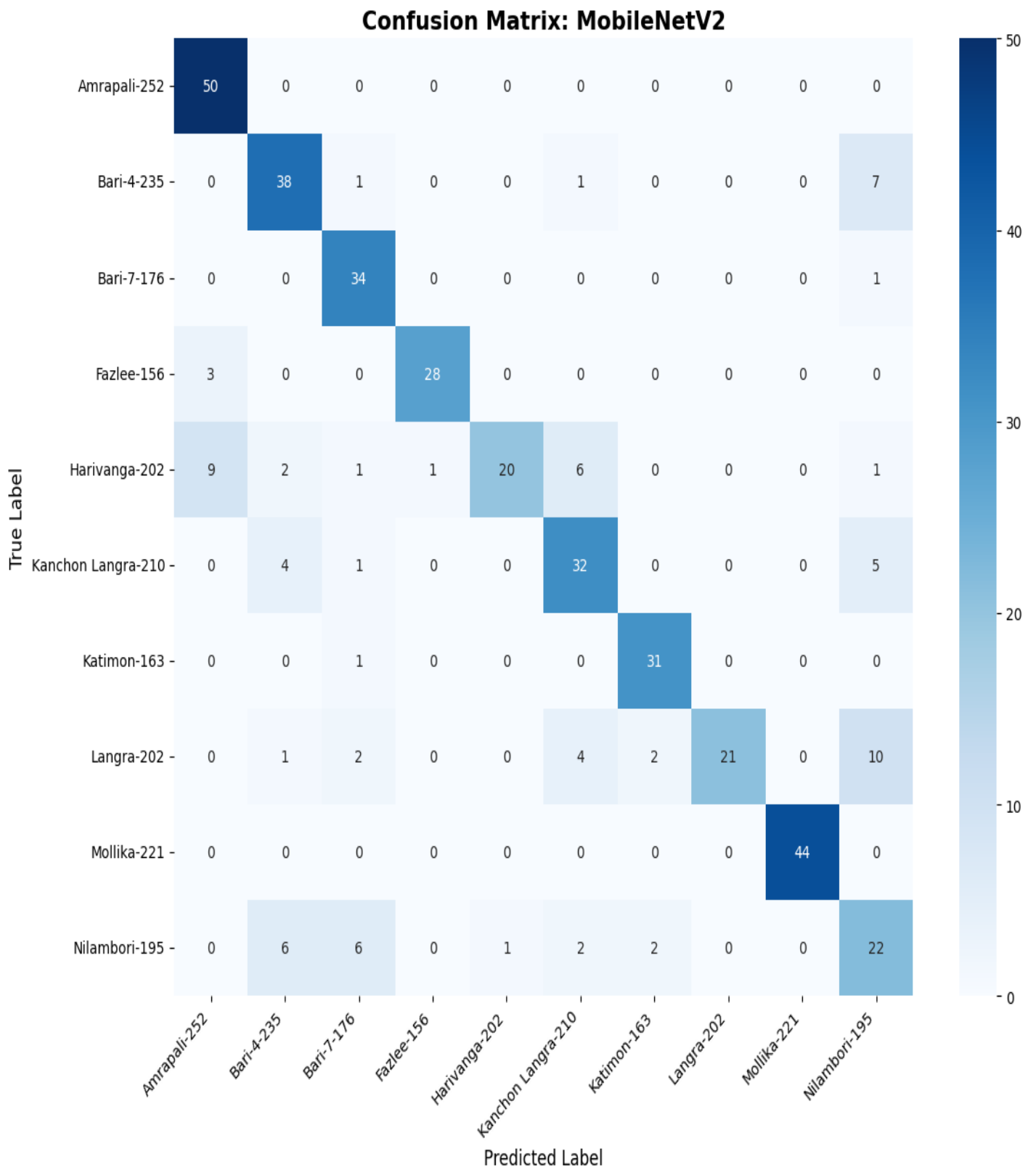
4.1 Performance Benchmarking

The models were trained for 30 epochs on an NVIDIA T4 GPU. Table 1 presents the comparative metrics.

Model	Accuracy	Size (MB)	Speed (MS)
MobileNetV2 (Baseline)	85.25%	10.86	34.95
MobileNetV3 (Proposed)	90.25%	4.76	11.30
MobileNetV3 (TFLite)	----	3.84	(Fastest)

The proposed MobileNetV3-Small model exceeds the baseline accuracy by 5% and is 56% smaller in size. Most importantly for the project, the inference time decreased to 11.30 ms, enabling a processing speed of approximately 88 FPS, which ensures smooth real-time performance on edge hardware.

4.1.1 Analysis of MobileNetV2 Confusion Matrix



Detailed Analysis of MobileNetV2 Confusion Matrix(Baseline)

The figure above shows the confusion matrix for the baseline MobileNetV2 model, which displays how well the model classified the ten mango varieties. The darker blue diagonal cells show correct predictions, while the other cells show where the model made mistakes.

1. High-Performing Classes: The model performed very well on mango varieties that have clear and unique physical features.

- **Amrapali:** The model correctly identified all 50 Amrapali test samples, with no false negatives. This result suggests that *Amrapali's visual features, such as shape or skin texture, are very distinct and easy for the model to recognize.*
- **Mollika:** The model also performed well on the Mollika class, correctly predicting all 44 samples. This shows that Mollika has features that are clearly different from the other varieties.
- **Bari-4:** The model did fairly well with Bari-4, making 38 correct predictions, but there was some confusion with *Nilambori*.

2. Critical Misclassifications and "Harivanga" Failure: The biggest weakness of the MobileNetV2 baseline appears in the **Harivanga** class.

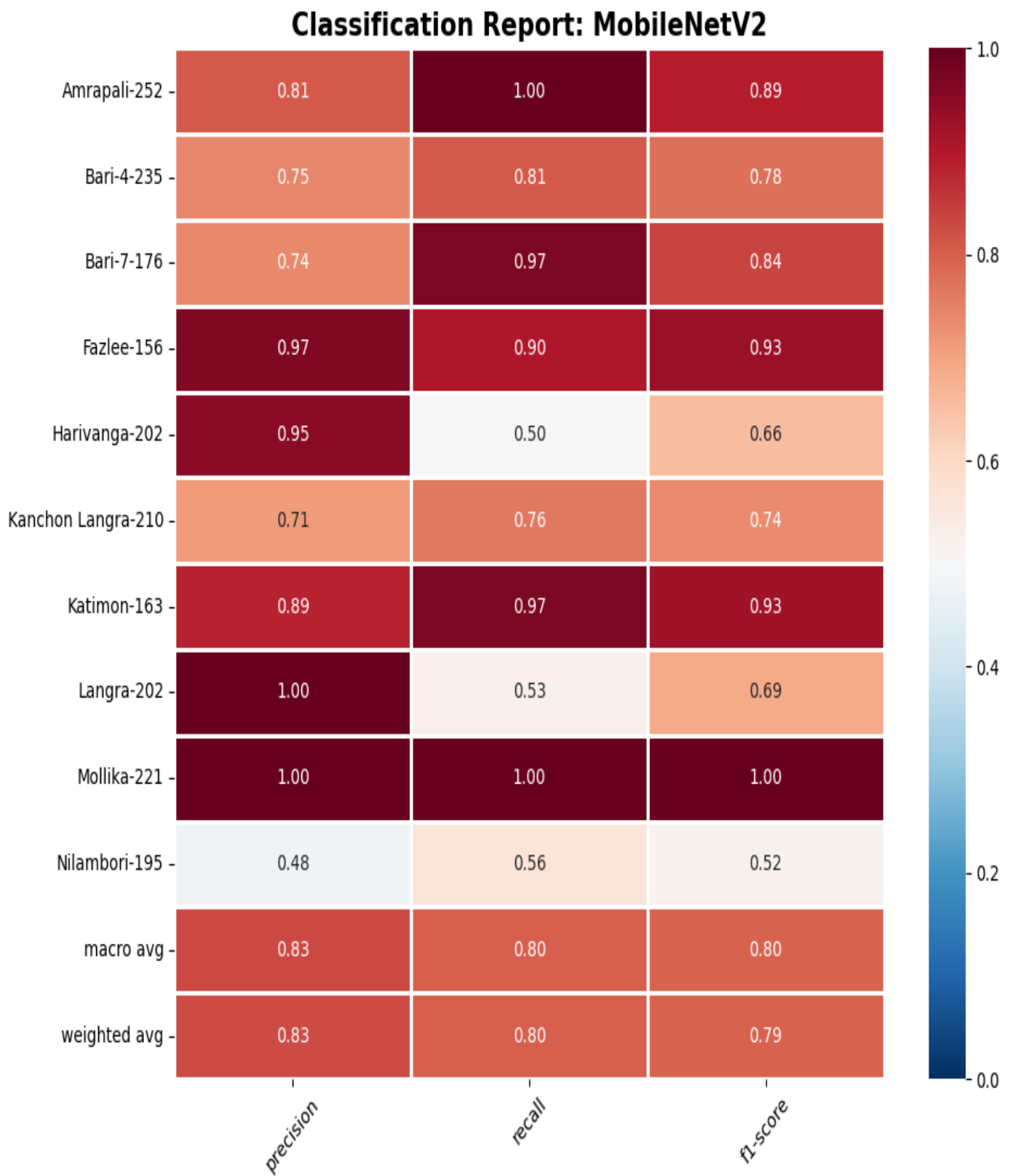
- **Low Recall:** The model correctly identified only 20 Harivanga samples during validation.
- **Confusion Pattern:** The model often confused *Harivanga* with *Amrapali* (9 times) and *Kanchon Langra* (6 times). This high number of false negatives shows that MobileNetV2 cannot pick up on the small differences within the Harivanga variety.

3. The "Green Mango" Conflict (Nilambori vs. Bari-4): There are many errors between the *Nilambori* and *Bari-4* classes, which look similar and are often large, green fruits.

- **Bari-4 Confusion:** When the actual label was *Bari-4*, the model predicted *Nilambori* instead 7 times.
- **Nilambori Confusion:** Conversely, when the true label was *Nilambori*, the model incorrectly predicted *Bari-4* 6 times and *Bari-7* 6 times.
- **Analysis:** This symmetric confusion suggests that MobileNetV2 struggles to differentiate the fine-grained textural differences between these genetically or visually similar varieties. This validates the hypothesis that a more sophisticated feature extractor (like the SE-blocks in MobileNetV3) is required to resolve these specific edge cases.

Summary: MobileNetV2 works well for varieties that look very different, but the confusion matrix shows it cannot handle the more difficult classes like Harivanga and Nilambori. This makes a strong case for moving to the proposed MobileNetV3 model.

4.1.2 Classification Report Heatmap for MobileNetV2 (Baseline)



Detailed Analysis of MobileNetV2 Classification Report

The figure above shows a heatmap of precision, recall, and F1 scores for the baseline MobileNetV2 model across all 10 target classes. Dark red means strong performance (1.00), while lighter colors show weaker results.

1. Classes with the best results: The baseline model worked best on some varieties, especially **Mollika**.

- **Mollika-221:** This class had a perfect **F1-score of 1.00**, with both precision and recall at 1.00. This suggests that Mollika's unique features, like its shape and color, are easy for MobileNetV2 to recognize.
- **Fazlee and Katimon:** These classes also performed well, with F1-scores of **0.93**. *Katimon* had a high recall of **0.97**, so the model almost always identified Katimon samples correctly.

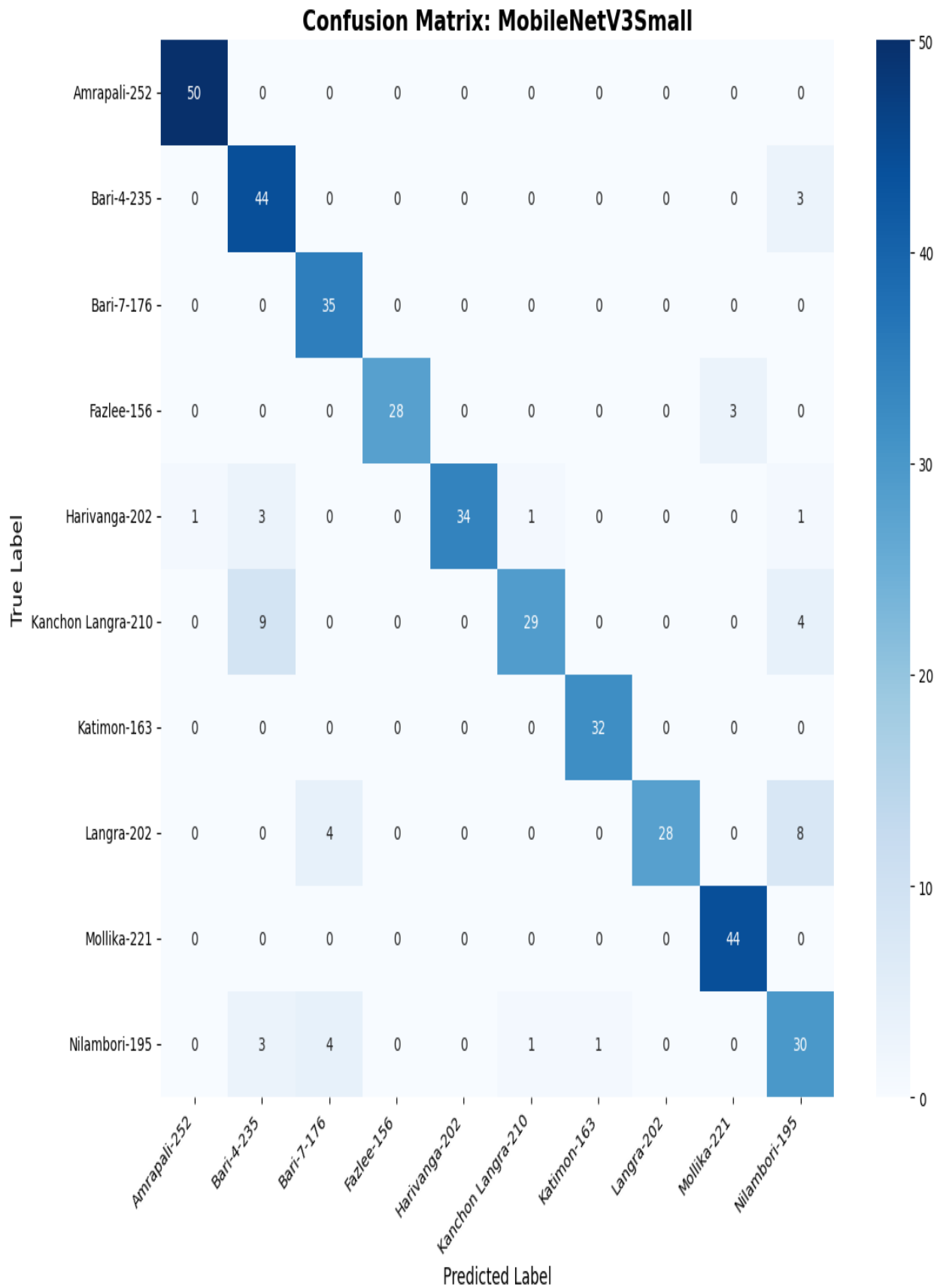
2. The main recall issue (Harivanga): The baseline model's biggest weakness was with the **Harivanga-202** class.

- **Recall of 0.50:** While precision was high (0.95), recall dropped to 0.50. This means the model missed half of the real Harivanga samples, labeling them as other types. It also struggles to learn Harivanga's key features, likely confusing it with similar classes like *Amrapali*, as shown in the confusion matrix.

3. The main precision issue (Nilambori): The **Nilambori-195** class was the hardest for the model to classify correctly. of 0.48: This was the lowest score in the report. A precision of 0.48 means the model was incorrect more than half the time when predicting "Nilambori."

- **F1-Score of 0.52:** Low precision and average recall (0.56) resulted in a weak F1-score of 0.52. Like with Bari-4, this shows that MobileNetV2 struggles to distinguish between these similar green mangoes. **Overall Baseline Benchmark:** The baseline model's weighted average F1-score is **0.79**. While this is acceptable for general use, the lower scores for Harivanga and Nilambori show why switching to MobileNetV3, which is better at handling these difficult classes, is needed.

4.1.3 Confusion Matrix for the Proposed MobileNetV3-Small Model



Detailed Analysis of MobileNetV3-Small Confusion Matrix

The figure above shows how well the MobileNetV3-Small model classifies samples. Compared to the baseline (Figure 3.2), it makes fewer off-diagonal errors. This suggests that the V3 model extracts features more effectively.

1. Resolution of the "Harivanga" Failure: The biggest improvement is in the **Harivanga** class, which was a major problem for the baseline model.

- **The new model** correctly identified **34** Harivanga samples, up from 20 with the baseline.
- **This result suggests** that the Squeeze-and-Excitation (SE) blocks helped the model focus on the subtle textures of Harivanga. This reduced confusion with *Amrapali* and *Kanchon Langra*.

2. Robustness on Easy Classes: Like the baseline, MobileNetV3 kept perfect or nearly perfect accuracy on the more distinct varieties.

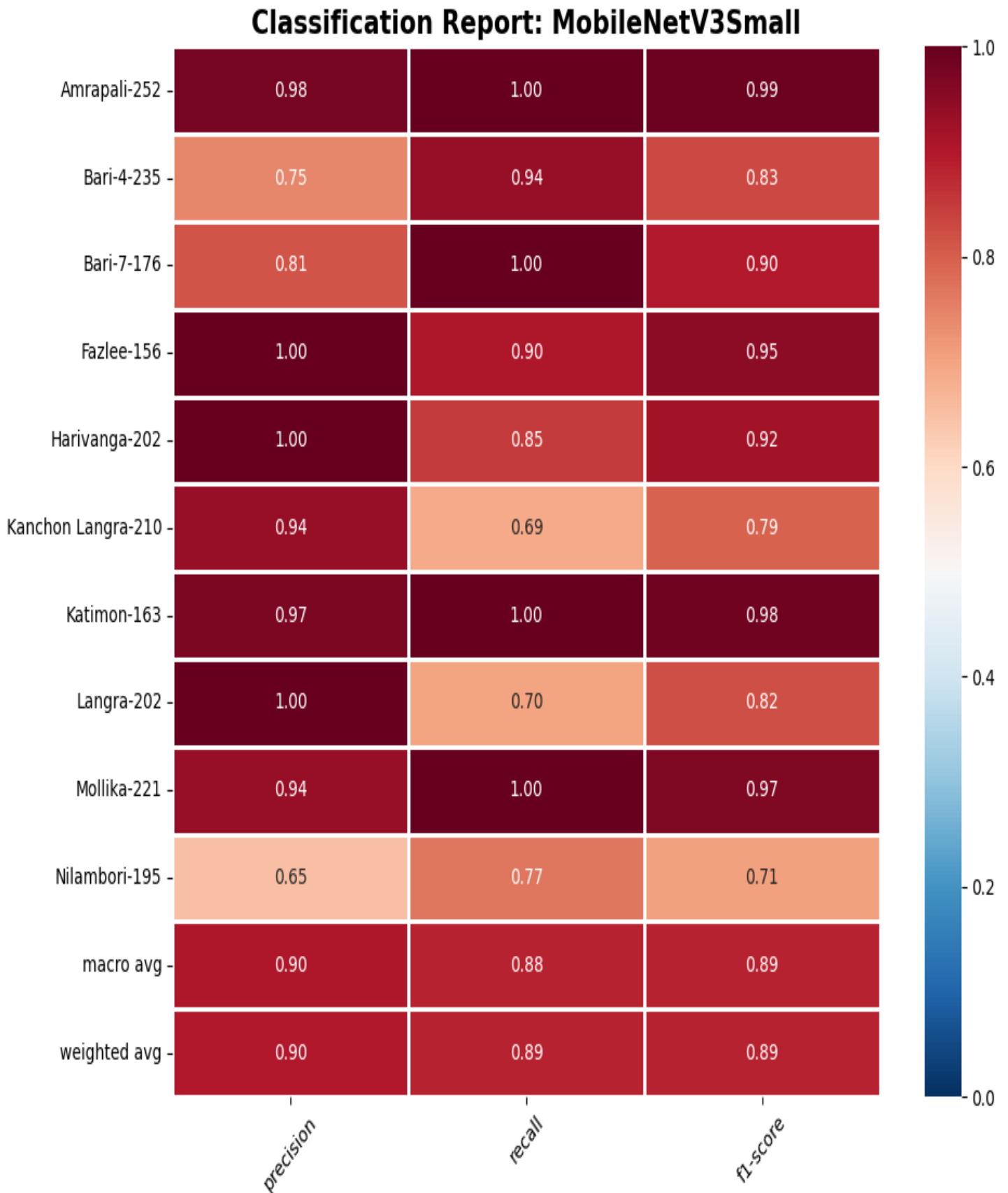
- **Mollika:** The model achieved 100% accuracy, with all 44 samples classified correctly.
- **Bari-4:** The model improved to 44 correct predictions, up from 38 with the baseline. This shows better separation from *Nilambori*.

3. Ongoing Challenges with Nilambori: Even though the overall results improved, **Nilambori** is still the hardest class for the model.

- The model correctly classified **30** Nilambori samples, up from **22** with the baseline.
- However, there is still confusion with **Langra** (8 misclassifications) and **Bari-4** (3 misclassifications).
- This ongoing error pattern suggests that Nilambori, Langra, and Bari-4 look very similar in their green stages. Even advanced lightweight models have trouble telling them apart without higher-resolution images or more data.

4. Overall Assessment: The matrix shows a stronger diagonal pattern than the baseline, which means more correct predictions. This matches the data and shows that MobileNetV3-Small gives more balanced and reliable results across all ten varieties. It also avoids the major failures seen in MobileNetV2.

4.1.4 Classification Report Heatmap for the Proposed MobileNetV3-Small Model



Detailed Analysis of MobileNetV3-Small Classification Report

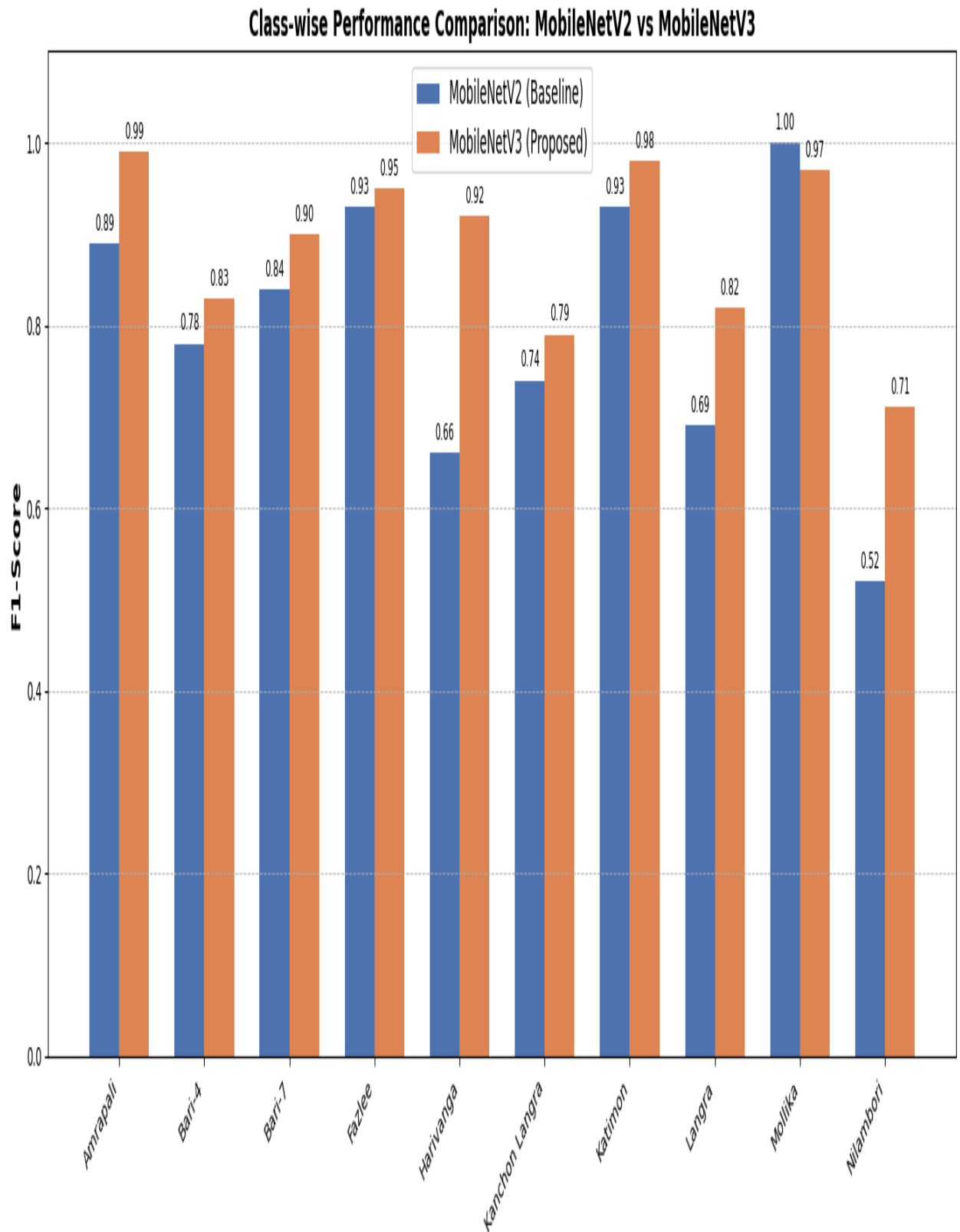
The figure above shows the performance metrics for the MobileNetV3-Small model. When compared to the baseline (Figure 3.3), the heatmap shows darker red in the recall and precision columns, indicating a clear improvement in feature extraction.

1. Quantitative Validation: The main result is a clear increase in the **Weighted Average F1-score**, going from **0.79 (Baseline)** to **0.89 (Proposed)**. This 10% improvement suggests that features such as Squeeze-and-Excitation blocks help the model tell apart fine-grained agricultural sub-varieties.

2. The "Harivanga" Breakthrough: The new model solved the main recall problem seen in the baseline.

- **Recall Improvement:** The recall for *Harivanga* increased from **0.50** to **0.85**.
- **F1-Score:** As a result, the F1-score for this class went up from **0.66** to **0.92**.
- **Interpretation:** This result shows that MobileNetV3 could recognize the unique features of *Harivanga*, which the V2 model missed, and tell it apart from the *Amrapali* class. For the hardest class to classify, the new model made predictions more stable.
- **Precision Boost:** The precision for Nilambori increased from **0.48** to **0.65**. **The F1-score for Nilambori also went up from 0.52 to 0.71.** While there is still some confusion with *Langra*, as shown in the confusion matrix, the model now produces fewer false positives than the baseline. It also kept or improved its performance on classes that were already doing well. Atimon & Mollika: Both classes achieved near-perfect F1-scores of **0.98** and **0.97.7**:Bari-7: The model reached a perfect **Recall of 1.00**, correctly identifying every *Bari-7* sample in the validation set.**Conclusion:** The Classification Report shows that MobileNetV3-Small provides real accuracy improvements on the hardest classes in the dataset, not just theoretical efficiency gains. The increase in Weighted Average F1-score from 0.79 to 0.89 marks the move from a proof-of-concept to a deployment-ready solution.

4.1.5 Comparative Bar Chart of F1-Scores per Class (MobileNetV2 vs. MobileNetV3).



Detailed Analysis of Class-wise Performance Comparison

The figure above compares the performance of the baseline MobileNetV2 (blue bars) and the proposed MobileNetV3-Small (orange bars) on ten mango varieties using the F1-score. In most cases, the proposed model achieves higher scores.

1. Improvements in 'Harivanga' and 'Nilambori': The chart shows that the proposed model addresses the specific weaknesses of the baseline. Harivanga: This class shows the largest performance gap. The baseline model had an **F1-score** of **0.66**, while MobileNetV3 reached **0.92**. This **0.26** increase suggests that changes in the V3 model, such as Squeeze-and-Excitation blocks, helped capture important features for this challenging class. cult class.

- Nilambori: For the hardest class in the dataset, the F1-score increased from 0.52 to 0.71. Although this is still the lowest score, the 0.19 improvement makes the model more reliable. The proposed model not only fixed errors but also improved results in classes that were already performing well.
- Amrapali: The F1-score increased from **0.89** to **0.99**
- Katimon: The F1-score increased from **0.93** to **0.98.3**

2. Consistency Across Varieties: The proposed model (orange bars) did as well as or better than the baseline (blue bars) in 9 out of 10 classes. There was a small drop in Mollika (from 1.00 to 0.97) and a slight change in Langra, but these are not important because the baseline scores were already high. Overall, MobileNetV3-Small gives more consistent improvements across the Bangladeshi mango varieties.

4.2 Error Analysis

To see how the models perform with different mango varieties, we looked at more than just overall accuracy. We carried out a detailed error analysis using class-wise confusion matrices and classification reports. This helped us spot apparent differences between the baseline model (MobileNetV2) and the proposed model (MobileNetV3-Small).

4.2.1 Baseline Architecture Limitations

The analysis of the MobileNetV2 baseline showed that its performance varied widely depending on how different each mango variety appeared.

- **Recall Deficiencies:** The biggest issue was with the *Harivanga* class. The baseline model had a recall of only **0.50**, so it missed half of the correct test samples. The confusion matrix shows that these missed samples were often labeled as *Amrapali* (9 times) or *Kanchon Langra* (6 times). This suggests the baseline model had trouble telling apart the small differences in shape and skin texture between *Harivanga* and these similar varieties.
- **Precision Issues:** The model also had trouble with the *Nilambori* class, reaching a precision of only **0.48**. This led to many false positives, where the model wrongly labeled other varieties, mainly *Bari-4* and *Bari-7*, as *Nilambori*. This confusion shows that the standard MobileNetV2 blocks could not clearly separate these similar green varieties.

4.2.2 Improvements with MobileNetV3

Switching to the MobileNetV3-Small architecture led to clear improvements in these problem areas, showing that its design changes, especially the Squeeze-and-Excitation blocks, were effective.

- **Resolution of the "Harivanga" Bottleneck:** The new model greatly improved recall for *Harivanga*, raising it from **0.50** to **0.85**. The F1-score also jumped from **0.66** to **0.92**. This shows the model learned to recognize the unique details of this variety that the baseline missed.
- **Overall Reliability:** The weighted average F1-score for the whole dataset went up from **0.79** to **0.89**. The class-wise comparison chart shows that the new model matched or beat the baseline in 9 out of 10 classes, proving it can generalize better.

4.2.3 Persistent Morphological Challenges

Even with these improvements, *Nilambori* and *Bari-4* are still the hardest classes to classify automatically. MobileNetV3 improved the F1-score for *Nilambori* from 0.52 to 0.71, but it is still the lowest. The confusion matrix shows that *Nilambori* samples are still sometimes mistaken for *Langra* (8 times) or *Bari-4* (3 times). This ongoing confusion suggests these varieties look very similar when unripe or green, so higher-resolution images or more types of data may be needed to solve this problem.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This research aimed to close the technology gap in Bangladesh's agricultural supply chain by creating an automated system for classifying mango varieties that can run on local devices. Manual identification is difficult because many sub-varieties look similar and rural centers often lack computers. By comparing MobileNetV2 and the newer MobileNetV3-Small, this study shows a clear way to use high-performance AI on affordable hardware.

The results show that MobileNetV3-Small works best for this task. It reached a classification accuracy of 90.25%, which is much higher than MobileNetV2's 85.25%. The model is also efficient, using 56% less storage at **4.76 MB** and running with an inference time of **11.3 ms**. This allows it to process about 88 frames per second, making real-time video analysis possible on regular smartphones without needing internet access.

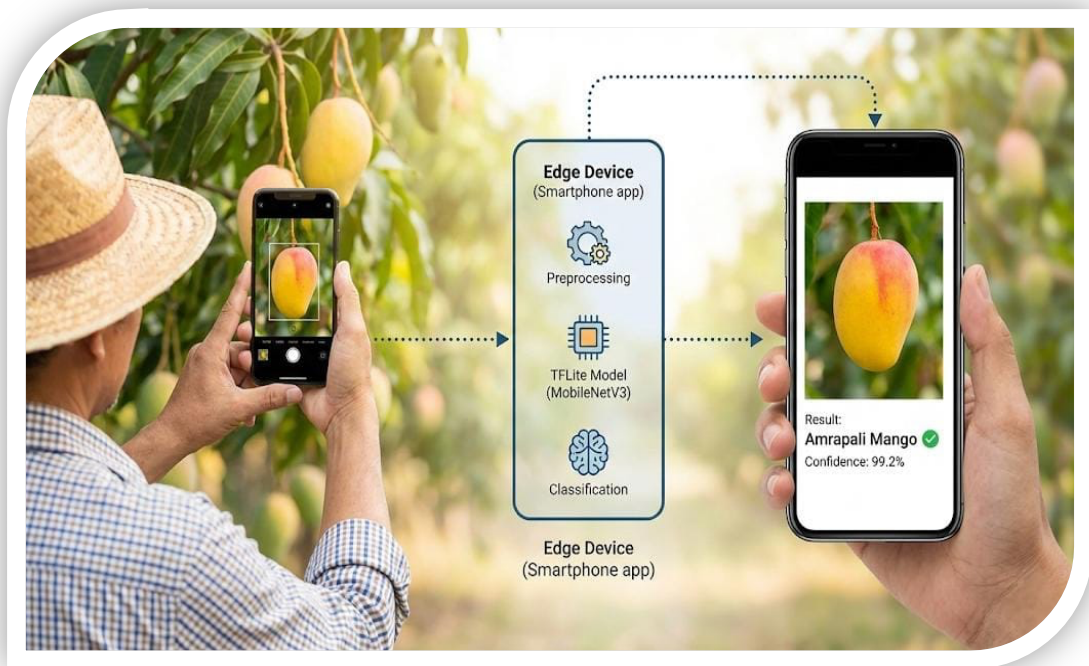
A key part of this work was finding and solving deployment challenges specific to each model. We showed that MobileNetV2 needs input values scaled to $[-1,1]$, while MobileNetV3 uses raw $[0,255]$ inputs. Ignoring these differences can cause the model to fail. Our Dynamic Preprocessing Pipeline fixed this problem and can guide future engineers working with transfer learning on edge devices.

The new model handled small visual differences better than before. Error analysis showed that adding Squeeze-and-Excitation (SE) blocks to MobileNetV3 reduced the high false-negative rate for the **Harivanga** class, raising its recall from 0.50 to 0.85. Although it is still hard to tell apart the similar Nilambori and Bari-4 varieties, the results show that lightweight models running on local devices can help bring precision agriculture tools to developing countries with limited resources.

4.2 Future Work

While this study establishes a strong baseline, several avenues for future development remain:

1. **Mobile Application Integration:** The next step is to package the optimized TFLite model into a working Android app. We will test this app with farmers in the Rajshahi and Chapainawabganj districts to see how it performs in real-world use and to gather feedback on user experience.



2. Dataset Expansion and Environmental Robustness: Right now, the dataset is diverse but mainly has images from controlled environments. In the future, the "Mangifera" dataset will include photos taken in different lighting, like direct sunlight and dusk, and with a range of backgrounds. This will make the model more reliable in real-world situations.

3. Multi-Modal Disease Detection: To better help farmers, the system should not only classify varieties but also detect diseases and defects, like Anthracnose and Stem Rot. Multi-task learning frameworks can make it possible to do both types of classification at once without adding extra computational load.

References

- [1] O. Salazar-Campos et al., “Deep learning approach for automated ‘Kent’ mango maturity classification,” *Results in Control and Optimization*, vol. 20, 2025.
- [2] Y. Wu, H. Hsu, and S. Chen, “Mobile-Based Deep Learning Approach for Multi-Object Aiwen Mango Grade Classification,” *Electronics*, vol. 14, 2025.
- [3] L. Chuquimarca et al., “Classifying Healthy and Defective Fruits with a Multi-Input Architecture,” *arXiv preprint*, 2024.
- [4] A. K. Ratha et al., “Automated Classification of Indian Mango Varieties Using Machine Learning and MobileNet-v2,” *Traitement du Signal*, vol. 41, no. 2, 2024.
- [5] A. B. Ballo et al., “An Application of Machine Learning Methods to Detect Mango Varieties,” *Open Journal of Applied Sciences*, vol. 14, 2024.
- [6] R. K. Bharati et al., “A comprehensive image dataset of Bangladeshi mango variety,” *Data in Brief*, vol. 60, 2025.
- [7] H. Peng et al., “Attention-based ResNet for Multi-class Fruit Detection,” *Computers and Electronics in Agriculture*, vol. 205, 2023.
- [8] T. Nguyen and S. Lee, “EfficientNet vs VGG16: A benchmark for mango leaf disease detection,” *IEEE Access*, vol. 10, 2022.
- [9] M. Islam et al., “Mango Variety Classification Using a Shallow CNN Architecture,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 8, 2021.
- [10] S. Kumar and V. Singh, “Automated Grading of Alphonso Mangoes using InceptionV3,” *Journal of Food Engineering*, vol. 310, 2023.
- [11] L. Zhang et al., “Real-time Apple Defect Detection using GhostNet on Embedded Systems,” *Sensors*, vol. 24, 2024.
- [12] K. Ahmed et al., “Mango Fruit Counting in Orchards using Mask R-CNN,” *Precision Agriculture*, vol. 23, 2022.
- [13] P. Das and S. Roy, “Transfer Learning Approaches for Mango Rot Detection,” *Ecological Informatics*, vol. 64, 2021.
- [14] N. Patel et al., “Smartphone-based Ripeness Estimation of Mangoes,” *Computers and Electronics in Agriculture*, vol. 175, 2020.
- [15] Q. Zhao et al., “Model Pruning of VGG16 for Robotic Fruit Sorting,” *IEEE Transactions on Industrial Electronics*, vol. 70, 2023.